

Traivo — Orderkoncept Gap-Analys

Datum: 2026-06-16

Källa: Skärmbild “Skapa Orderkoncept” + Mats sessionstranskript (session_2026-06-16T16-56.json)

Syfte: Identifiera vad som saknas i traivo-one/traivo-go för att realisera det nya 7-steps orderkonceptet

1. Sammanfattning av det nya orderkonceptet

Orderkonceptet är ett **7-steps wizard-flöde** för att skapa komplexa, återanvändbara ordermallar:

Steg	Namn	Beskrivning
1	Namn & Kund	Namnge orderkonceptet + välj fast kund ELLER kund från objektmetadata
2	Pris & Referens	Prislista, fast pris per uppdrag, e-referens, e-beteckning
3	Fakturering	Faktureringsnivå (metadata-villkor), metod (Avrop/Schema/Abonnemang), period
4	Inpekning	Sök & välj objekt (kluster/grenar/områden/enskilda), villkorsfilter
5	Leveranstid	Datum, klockslag, upprepade scheman
6	Uppgifter	Artiklar/uppgifter kopplade till ordern med metadata-villkor
7	Granska	Sammanfattning + bekräftelse

UI-element från skärmbilden

- **Header:** “Skapa Orderkoncept” med steg-indikator (progress stepper)
- **Namnfält:** Textinput med placeholder “T.ex. Sophämtning Centrum 2026”
- **Kundval:** Radio-knappar — “Fast kund” / “Från objektets metadata”
- **Kundsökning:** Sökfält + lista (visar “Axfood Sverige”)

- **Sammanfattningspanel (höger):** Objekt (0), Artiklar (0), Värde (0 kr), Kostnad (0 kr), Tid (0.0 h)
 - **Navigation:** Tillbaka-knapp, "Spara utkast", "Nästa →"
-

2. Nuvarande implementation i traivo-one/traivo-go

Baserat på tidigare analys av repositories:

Befintlig ordermodell

- Enkel orderhantering med **4-steps statusflöde:** Dispatched → On Way → On Site → Completed
- Grundläggande orderfält: titel, beskrivning, status, tilldelad tekniker
- Koppling till ett enskilt objekt/kund
- Ingen konceptlager / mallsystem ovanpå ordrar

Befintlig datamodell (Drizzle ORM / PostgreSQL)

- `companies` — multi-tenant
- `users` — medarbetare/tekniker
- `customers` — grundläggande kunddata
- `objects` — serviceobjekt
- `orders` — enskilda ordrar (enkel modell)
- `articles` — artikelregister (grundläggande)

Saknas helt

- Ingen "orderkoncept"-entitet
 - Ingen wizard/stepper-UI för orderskapande
 - Ingen prislistehantering
 - Ingen faktureringsmodul
 - Ingen metadata-villkorsmotor
 - Ingen hierarkisk objektstruktur (kluster → grenar → områden → enskilda objekt)
-

3. Identifierade GAP

3.1 📁 Datamodeller som behöver läggas till

Modell	Beskrivning	Prioritet
order_concepts	Huvudentitet för order-koncept (namn, status, draft/active, company_id)	 Kritisk
or- der_concept_customer_config	Kundkonfiguration: typ (fast/metadata), customer_id, metadata_field_key	 Kritisk
order_concept_pricing	Prissättning: price_list_id, fixed_price, price_per_assignment	 Kritisk
order_concept_invoice_refs	E-referens, e-beteckning för fakturahuvud	 Hög
order_concept_billing	Faktureringsnivå, metod (avrop/schema/abonnemang), period, startdatum	 Kritisk
or- der_concept_billing_conditions	Villkor för fakturanivå (metadata-fält + operator + värde)	 Hög
order_concept_targeting	Inpekning: valda objekt/kluster/grenar med villkor	 Kritisk
or- der_concept_targeting_conditions	Villkorsfilter per inpekning (metadata + operator + värde)	 Hög
order_concept_delivery	Leveranstid: datum, tid, upprepningsschema	 Hög
order_concept_tasks	Kopplade artiklar/uppgifter med metadata-villkor	 Kritisk
price_lists	Prislistor med artikelpriser	 Hög
price_list_items	Enskilda priser per artikel i en prislista	 Hög
object_hierarchy	Hierarkisk struktur: kedja → kluster → gren → område → objekt	 Kritisk
object_metadata	Nyckel-värde metadata per objekt (flexibelt schema)	 Kritisk

Modell	Beskrivning	Prioritet
metadata_field_definitions	Definition av tillgängliga metadatafält per company	● Hög
invoices	Fakturaentitet med status, period, belopp	● Hög

Förslag: Drizzle-schema för `order_concepts`

```
// schema/orderConcepts.ts
import { pgTable, uuid, text, varchar, timestamp, numeric, boolean, jsonb } from 'drizzle-orm/pg-core';

export const orderConcepts = pgTable('order_concepts', {
  id: uuid('id').primaryKey().defaultRandom(),
  companyId: uuid('company_id').notNull().references(() => companies.id),
  name: varchar('name', { length: 255 }).notNull(),
  status: varchar('status', { length: 50 }).default('draft'), // draft, active, archived

  // Steg 1: Kund
  customerType: varchar('customer_type', { length: 20 }), // 'fixed' | 'metadata'
  fixedCustomerId: uuid('fixed_customer_id').references(() => customers.id),
  customerMetadataField: varchar('customer_metadata_field', { length: 100 }),

  // Steg 2: Pris & Referens
  priceListId: uuid('price_list_id').references(() => priceLists.id),
  fixedPricePerAssignment: numeric('fixed_price_per_assignment'),
  eReference: varchar('e_reference', { length: 255 }),
  eDesignation: varchar('e_designation', { length: 255 }),

  // Steg 3: Fakturering
  billingMethod: varchar('billing_method', { length: 20 }), // 'avrop' | 'schema' | 'abonnemang'
  billingPeriod: varchar('billing_period', { length: 20 }), // 'daily' | 'weekly' | 'monthly' | 'quarterly'
  billingStartDate: timestamp('billing_start_date'),
  invoiceGrouping: varchar('invoice_grouping', { length: 20 }), // 'per_assignment' | 'weekly' | 'monthly'
  billingLevelConditions: jsonb('billing_level_conditions'), // [{field, operator, value}]

  // Steg 5: Leveranstid
  deliveryDate: timestamp('delivery_date'),
  deliveryTimeStart: varchar('delivery_time_start', { length: 5 }),
  deliveryTimeEnd: varchar('delivery_time_end', { length: 5 }),
  repeatSchedule: jsonb('repeat_schedule'), // cron-liknande schema

  // Metadata
  createdAt: timestamp('created_at').defaultNow(),
  updatedAt: timestamp('updated_at').defaultNow(),
  createdBy: uuid('created_by').references(() => users.id),
});

export const orderConceptTargets = pgTable('order_concept_targets', {
  id: uuid('id').primaryKey().defaultRandom(),
  orderConceptId: uuid('order_concept_id').notNull().references(() => orderConcepts.id),
  objectId: uuid('object_id').references(() => objects.id),
  hierarchyNodeId: uuid('hierarchy_node_id'), // kluster/gren/område
  hierarchyLevel: varchar('hierarchy_level', { length: 20 }), // 'chain' | 'cluster' | 'branch' | 'area' | 'object'
  conditions: jsonb('conditions'), // [{field, operator, value}]
});

export const orderConceptTasks = pgTable('order_concept_tasks', {
  id: uuid('id').primaryKey().defaultRandom(),
  orderConceptId: uuid('order_concept_id').notNull().references(() => orderConcepts.id),
  articleId: uuid('article_id').notNull().references(() => articles.id),
  sortOrder: integer('sort_order').default(0),
});
```

```
metadataConditions: jsonb('metadata_conditions'), // [{field, operator, value}]  
});
```

3.2 🚧 API-endpoints som saknas

Endpoint	Metod	Beskrivning	Steg
/api/order-concepts	GET	Lista orderkoncept (med paginering, filter)	—
/api/order-concepts	POST	Skapa nytt orderkoncept (draft)	1
/api/order-concepts/:id	GET	Hämta enskilt orderkoncept	—
/api/order-concepts/:id	PATCH	Uppdatera orderkoncept (stegvis sparning)	1-7
/api/order-concepts/:id	DELETE	Ta bort orderkoncept	—
/api/order-concepts/:id/activate	POST	Aktivera orderkoncept (draft → active)	7
/api/order-concepts/:id/draft	POST	Spara utkast	1-6
/api/order-concepts/:id/targets	GET/POST/DELETE	Hantera inpekade objekt	4
/api/order-concepts/:id/targets/preview	POST	Förhandsgranska matchade objekt	4
/api/order-concepts/:id/tasks	GET/POST/PUT/DELETE	Hantera kopplade artiklar/uppgifter	6
/api/order-concepts/:id/summary	GET	Sammanfattning (objekt, artiklar, värde, kostnad, tid)	7
/api/customers	GET	Söka kunder (med sökfält)	1
/api/customers/:id/metadata	GET	Hämta kundmetadata	1
/api/price-lists	GET/POST	Prislistor CRUD	2

Endpoint	Metod	Beskrivning	Steg
/api/price-lists/:id/items	GET/POST	Artiklar i prislista	2
/api/objects/hierarchy	GET	Hämta objekthierarki (träd)	4
/api/objects/search	GET	Sök objekt (Hemköp, pantkärl etc.)	4
/api/objects/:id/metadata	GET	Hämta metadata per objekt	4
/api/metadata-fields	GET	Lista tillgängliga metadatafält	1,3,4
/api/articles/search	GET	Sök artiklar att koppla till koncept	6
/api/order-concepts/:id/test-conditions	POST	Testa villkorsfilter mot specifikt objekt	4

3.3 UI-komponenter som behöver utvecklas

Komponent	Beskrivning	Komplexitet
OrderConceptWizard	7-steps wizard med progress stepper	 Hög
StepIndicator	Horisontell steg-indikator (1-7) med aktiv/klar-marker-ing	 Låg
SummaryPanel	Höger sidebar: Objekt, Artiklar, Värde, Kostnad, Tid	 Medium
Step1_NameAndCustomer	Namnfält + kundval (radio: fast/metadata) + kund-sökning	 Medium
CustomerSearch	Sökfält med autocomplete-lista för kunder	 Medium
MetadataFieldPicker	Dropdown för att välja metadatafält (kund, kundnr etc.)	 Medium
Step2_PriceAndReference	Prislista-val, fast pris, e-referens, e-beteckning	 Medium
PriceListSelector	Sök + välj prislista, alternativt ange fast pris	 Medium
Step3_Billing	Faktureringsnivå (villkor) + metod + period	 Hög
BillingMethodSelector	Radio/select: Avrop, Schema, Abonnemang	 Låg
BillingPeriodConfig	Faktureringsperiod + startdatum (vid abonnemang)	 Medium
InvoiceGroupingSelector	Per uppdrag / vecko / månads-samlingsfaktura	 Medium
Step4_Targeting	Objektträd med sök, kollaps/expansion, flerval, villkor	 Hög
ObjectHierarchyTree	Expanderbart träd: kedja → kluster → gren → område → objekt	 Hög
ObjectSearch		 Medium

Komponent	Beskrivning	Komplexitet
	Sökfält som filtrerar i objekt-trädet	
MultiSelectCheckboxes	Flerval vid kollaps/expansion-sknapp	● Medium
ConditionBuilder	Villkorsbyggare: metadatafält + operator + värde	● Hög
ConditionTestPanel	Testruta: välj objekt → se om villkor matchar	● Medium
Step5_DeliveryTime	Datum/tidsväljare, upprepningsschema	● Medium
Step6_Tasks	Artikelsökning + tillägg med metadata-villkor	● Medium
ArticleSearch	Sökfält + lista för att lägga till artiklar/uppgifter	● Medium
Step7_Review	Sammanfattning av alla steg, bekräftelse-knapp	● Medium
DraftSaveButton	“Spara utkast” — sparar aktuellt tillstånd utan validering	● Låg

3.4 Affärslogik som inte finns implementerad

A. Metadata-villkorsmotor (Condition Engine)

Den mest centrala saknade komponenten. Används i **steg 3, 4 och 6**.

Operatorer som behövs:

- lika med (equals)
- inte lika med (not equals)
- innehåller (contains)
- börjar med (starts with)
- större än (greater than)
- mindre än (less than)
- finns (exists / is not null)
- saknas (is null / does not exist)

Implementation:

```

interface Condition {
  field: string;      // metadatafälts nyckel
  operator: ConditionOperator;
  value?: string;     // ej nödvändigt för 'exists'/'missing'
}

type ConditionOperator =
  | 'equals' | 'not_equals'
  | 'contains' | 'starts_with'
  | 'greater_than' | 'less_than'
  | 'exists' | 'missing';

function evaluateCondition(condition: Condition, metadata: Record<string, string>): boolean;
function evaluateConditions(conditions: Condition[], metadata: Record<string, string>, logic: 'AND' | 'OR'): boolean;

```

B. Faktureringslogik

Funktion	Beskrivning
Avrop	Generera faktura när order slutförs (completed)
Schema	Fakturering post-arbete, samlad per period
Abonnemang	Periodisk fakturering (månad/kvartal) med startdatum
Samlad faktura	Gruppering per uppdrag / vecka / månad
Fakturaplacering	Hitta rätt nivå i hierarkin baserat på metadata-villkor (bubbla uppåt om villkor ej matchar)

C. Hierarkisk fakturarouting

Fakturan ska "bubbla uppåt" i objekthierarkin tills ett metadata-villkor matchar:

```

Objekt → Område → Gren → Kluster → Kedja → Kund (toppen)
      ↑ om villkor matchar → stoppa här

```

D. Orderkoncept-till-Order-generering

Logik för att expandera ett orderkoncept till enskilda ordrar:

- Matcha objekt via inpekning + villkor
- Koppla artiklar baserat på metadata-villkor
- Skapa individuella ordrar med rätt kund, pris och leveranstid

E. Prisberäkning

- Listpris: hämta pris från prislista per artikel
- Fast pris: ange ett fast pris per uppdrag
- Beräkna totalt värde/kostnad i sammanfattningspanelen

3.5 Konfigurationsparametrar som behövs

Parameter	Typ	Beskrivning
BILLING_METHODS	enum	['avrop', 'schema', 'abon-nemang']
BILLING_PERIODS	enum	['daily', 'weekly', 'monthly', 'quarterly']
INVOICE_GROUPING	enum	['per_assignment', 'weekly', 'monthly']
CONDITION_OPERATORS	enum	Se villkorsmotor ovan
CUSTOMER_TYPES	enum	['fixed', 'from_metadata']
HIERARCHY_LEVELS	enum	['chain', 'cluster', 'branch', 'area', 'object']
ORDER_CONCEPT_STATUSES	enum	['draft', 'active', 'archived']
MAX_CONDITIONS_PER_RULE	number	Max antal villkor per villkor-sregel
DEFAULT_BILLING_PERIOD	string	Standard faktureringsperiod
CURRENCY	string	Valuta (SEK)

4. Prioriterad implementation-roadmap

Fas 1: Grund (Sprint 1-2)

1.  Datamodell: `order_concepts` + migrering
2.  Datamodell: `object_metadata` + `metadata_field_definitions`
3.  API: CRUD för orderkoncept (draft/stegvis sparning)
4.  UI: Wizard-skelett med StepIndicator + navigation

Fas 2: Steg 1-2 (Sprint 2-3)

1. API + UI: Kundsökning + kundval (fast/metadata)
2. API + UI: MetadataFieldPicker
3. Datamodell: `price_lists` + `price_list_items`
4. API + UI: Prislista-val + fast pris + e-referens

Fas 3: Villkorsmotor + Fakturering (Sprint 3-4)

1. **Condition Engine** — central komponent (återanvänds i steg 3, 4, 6)

2. API + UI: Faktureringskonfiguration (metod, period, nivå)
3. Logik: Fakturanivå via hierarki-traversering

Fas 4: Inpekning (Sprint 4-5)

1. Datamodell: `object_hierarchy` (hierarkiskt träd)
2. API + UI: ObjectHierarchyTree med sök + flerval
3. API + UI: Villkorsfilter per inpekning
4. API + UI: Förhandsvisning / testfunktion

Fas 5: Leverans + Uppgifter + Granska (Sprint 5-6)

1. API + UI: Leveranstid med schema-repetition
2. API + UI: Artikelkoppling med metadata-villkor
3. API + UI: Granskningssteg med fullständig sammanfattning
4. Logik: Orderkoncept → ordrar (expansion)

Fas 6: Fakturering & Samlad faktura (Sprint 6-7)

1. Fakturagenereringsmotor (avrop/schema/abonnemang)
 2. Samlingsfaktura-logik
 3. Faktura-UI
-

5. Kända buggar/problem från Mats genomgång

#	Problem	Steg	Åtgärd
1	Metadatafält dyker inte upp vid "Från objektets metadata"	1	Kräver koppling objekt → metadata
2	Kollapsa enskild gren i objektträdet fungerar ej	4	Fixa tree-toggle per nod
3	Sökfält blockerar kollapsfunktionen	4	Separera sök- och kollaps-logik
4	Kan ej välja hel kedja (t.ex. "Hemköpskedjan i hela Sverige")	4	Lägg till flerval på alla hierarkinivåer
5	Abonnemang saknar fakturaperiod + startdatum	3	Lägg till period-select + date-picker
6	Schema vs Avrop verkar ha identisk logik	3	Klarifiera skillnad eller slå ihop
7	Kan ej testa villkorsfilter (testknapp fattas)	4	Implementera ConditionTestPanel

6. Arkitekturrekommendationer

Condition Engine som delad modul

Villkorsmotorn är den **mest återanvända komponenten** (steg 3, 4, 6 + framtida bruk). Bygg den som:

```

packages/
  condition-engine/
    src/
      types.ts           # Condition, Operator typer
      evaluator.ts       # evaluateCondition(), evaluateConditions()
      validator.ts       # validateCondition()
    components/
      ConditionBuilder.tsx # UI-komponent (React)
      ConditionRow.tsx
      OperatorSelect.tsx

```

Stegvis sparning (Draft Pattern)

Använd `PATCH /api/order-concepts/:id` med partiell JSON för att spara per steg utan fullständig validering. Fullständig validering sker vid steg 7 (aktivering).

Sammanfattningspanel (Real-time)

Panelen (Objekt, Artiklar, Värde, Kostnad, Tid) beräknas server-side via `/api/order-concepts/:id/summary` och uppdateras vid varje stegs sparning.

Denna gap-analys kan användas direkt som specifikation för implementation i Replit.